Implementing Generics-SOP Combinators in Polytropos

https://gemini.google.com/app/969a256bfe81aed9

User prompt: How can `hpure` and `hcollapse` from `generics-sop` be implemented within `polytropos`?

Response: In `generics-sop`, `NP f xs` is an inductive GADT indexed by a type-level list, so `hpure` and `hcollapse` operate by structural recursion over a singleton spine (`SListI`) or GADT constructors. In `polytropos` (`polytropos-core`), products and sums share a flat, type-erased runtime representation backed by `SmallArray`:

```
-- Conceptual layout in Polytropos.Core
data Rep (f :: Type -> Type) a = Rep
  { repTag  :: {-# UNPACK #-} !Word
  , repVals :: !(SmallArray (f Any))
  }

newtype RowRep (f :: Type -> Type) (r :: RowKind) (row :: k) a = RowRep
  { unRowRep :: Rep f a }
```

Because `RowRep` already parameterizes every slot by a functor `f :: Type -> Type` (mirroring `NP f` and `SOP f`), implementing `hcollapse`, `hpure`, and `hcpure` becomes an array operation over `repVals` combined with `AllFields` dictionary vectors synthesized by `Polytropos.Plugin`.

1. Implementing `hcollapse`

In `generics-sop`, `hcollapse` extracts values from a structure parameterized by the constant functor `K a` (equivalent to `Data.Functor.Const.Const a`).

In `polytropos`, a `RowRep (Const a) r row x` stores an unpacked `SmallArray (Const a Any)` in `repVals`. Because the second type parameter of `Const a Any` is phantom, every slot in the array already has the concrete runtime representation of `a`. Consequently:

1. **No typeclass constraints (`AllFields` / `SListI`) are required.**
2. **No `unsafeCoerce` (`noInlineUnsafeCo`) is required**—`getConst :: Const a Any -> a` safely unwraps each slot directly.
3. **It works uniformly on both `ProductRowT` (records) and `SumRowT` (variants)**, returning all field values for a product or the active constructor's payload slots for a sum.

```
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE ScopedTypeVariables #-}

import Data.Foldable (toList)
import Data.Functor.Const (Const (..))
import Data.Primitive.SmallArray ( indexSmallArray, sizeofSmallArray )
import Polytropos.Core (RowRep (..), repVals)

-- | Collapse a homogenous Const-wrapped RowRep into a list of values.
-- Runs in O(n) over the underlying SmallArray with zero typeclass overhead.
hcollapse :: RowRep (Const a) r row x -> [a]
hcollapse (RowRep rep) =
  let arr = repVals rep
      n   = sizeofSmallArray arr
      go i
        | i >= n    = []
        | otherwise = getConst (indexSmallArray arr i) : go (i + 1)
  in go 0

-- Or simply via Foldable on SmallArray:
hcollapseFold :: RowRep (Const a) r row x -> [a]
hcollapseFold (RowRep rep) = map getConst (toList (repVals rep))
```

2. Implementing `hcpure` (Constrained Heterogeneous Pure)

In `generics-sop`, `hcpure :: All c xs => Proxy c -> (forall a. c a => f a) -> NP f xs` walks the inductive `All c xs` constraint tree.

In `polytropos`, `AllFields ProductRowT row c` is solved in $O(1)$ by `Polytropos.Plugin`, which inspects `row` at compile time and emits a literal `SmallArray` of runtime dictionaries via `fieldDicts`. Each element in `fieldDicts @ProductRowT @row @c` holds the dictionary for `c` corresponding to that slot's field type (coerced to `Any`).

To implement `hcpure`, iterate across the dictionary array returned by `fieldDicts`, bring each slot's `Dict` into scope, evaluate the rank-2 callback, and pack the resulting `[f Any]` slots with `productRep`:

```
{-# LANGUAGE AllowAmbiguousTypes #-}
{-# LANGUAGE ConstraintKinds #-}
```

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE RankNTypes #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE TypeApplications #-}

import Data.Kind (Constraint, Type)
import Data.Primitive.SmallArray (indexSmallArray, sizeofSmallArray)
import GHC.Exts (Any)
import Polytropos.Core
  ( AllFields (..)
  , Dict (..)
  , ProductRowT
  , RowRep (..)
  , noInlineUnsafeCo
  , productRep
  )

-- | Populate a product RowRep using a constrained rank-2 builder.
-- Polytropos.Plugin synthesizes the fieldDicts SmallArray at compile time.
hcpure
  :: forall c row f a.
     AllFields ProductRowT row c
  => (forall x. c x => f x)
  -> RowRep f ProductRowT row a
hcpure fn =
  let dicts = fieldDicts @ProductRowT @row @c
      n     = sizeofSmallArray dicts
      go i
        | i >= n    = []
        | otherwise =
            case indexSmallArray dicts i of
              Dict -> noInlineUnsafeCo (fn @Any) : go (i + 1)
  in RowRep (productRep (go 0))
```

Note on Effectful Generation (`QuickCheck` / `Applicative`): If `f` is a composition or you are generating slots effectfully (analogous to `hsequence $ hcpure ...`), `traverse` over the dictionary indices produces `m (RowRep f ProductRowT row a)`—which is how `polytropos-quickcheck` implements `Arbitrary` for `Record` via `AllFields`.

3. Implementing Unconstrained `hpure`

For unconstrained `hpure :: (forall x. f x) -> RowRep f ProductRowT row a`, the rank-2 argument needs no per-field dictionary and can be instantiated directly at `Any` (`fn @Any :: f Any`). However, `RowRep` still needs to know **how many slots** exist in `row` at runtime so `productRep` allocates an array of the right length.

There are two ways to obtain the row arity in `polytropos`:

Approach A: Via `AllFields` with a Universal (`Top`) Constraint

Because `Polytropos.Plugin` solves `AllFields ProductRowT row c` for any constraint `c` whose instances hold for all fields in `row`, defining a universal typeclass `Top` allows `hcpure` to specialize directly into `hpure`:

```
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE MultiParamTypeClasses #-}

class Top a
instance Top a

hpure
  :: forall row f a.
     AllFields ProductRowT row Top
  => (forall x. f x)
  -> RowRep f ProductRowT row a
hpure fn =
  let n = sizeofSmallArray (fieldDicts @ProductRowT @row @Top)
  in RowRep (productRep (replicate n (fn @Any)))
```

Approach B: Via `FieldNames` / Known Arity

If you are working in a satellite package or without `AllFields` for `Top` (for example, using `FieldNames row` as in `polytropos-beam` or a known row length `KnownNat`), you can derive the slot count from the length of the field names list:

```
hpureFromNames
  :: forall row f a.
     FieldNames row
  => (forall x. f x)
```

```
  -> RowRep f ProductRowT row a
hpureFromNames fn =
  let n = length (fieldNames @row)
  in RowRep (productRep (replicate n (fn @Any)))
```

Summary Comparison: `generics-sop` vs. `polytropos`

Operation	`generics-sop` (`NP f xs`)	`polytropos` (`RowRep f ProductRowT row a`)
Runtime Spine	Linked GADT (`Nil`, `:*`)	Unpacked `SmallArray (f Any)` in `Rep`
`hcollapse`	$O(N)$ GADT pattern-matching requiring `SListI`	$O(N)$ `SmallArray` loop over `repVals`; **zero constraints**
`hcpure`	Inductive typeclass resolution (`All c xs`)	$O(1)$ plugin-emitted `fieldDicts` array + `productRep`
`hpure`	Inductive spine traversal (`SListI xs`)	`replicate n (fn @Any)` packed via `productRep`

Sources

1. generics-sop-intro.md - GitHub
2. polytropos-core-0.1.0.2/test/Main.hs — hdiff
3. polytropos-plugin: TcPlugin that emits literal ... - lachlanrussell.com
4. src/Polytropos/Beam.hs - hdiff - luite

User prompt: Please do show how to implement `hcliftA2` and `hsequence` from `generics-sop` within polytropos.

Response: In `polytropos`, products and sums share the same underlying Rep layout—an unpacked `Word` constructor tag (`0` for products, active constructor index for sums) paired with a flat `SmallArray (f Any)` payload vector. Because RowRep already parameterizes every slot by a type constructor `f :: Type -> Type`, `hcliftA2` and `hsequence` avoid inductive GADT recursion (`SListI`) and operate directly on `SmallArray`.

1. Implementing `hcliftA2` (and `hliftA2`)

In `generics-sop`, `hcliftA2` walks the inductive `All c xs` constraint list alongside two NP spines:

```
-- generics-sop signature:
hcliftA2 :: All c xs => Proxy c -> (forall a. c a => f a -> g a -> h a) -> NP f xs -> NP g xs -> NP h xs
```

In `polytropos`, `Polytropos.Plugin` solves `AllFields ProductRowT row c` in $O(1)$ at compile time by emitting a flat `SmallArray (Dict (c Any))` via `fieldDicts`. To implement `hcliftA2`, index in lockstep across three flat arrays of identical length n: the `fieldDicts` array, the first operand's `repVals`, and the second operand's `repVals`.

A. Zero-Intermediate-List Implementation (`runSmallArray`)

Using `runSmallArray` from `Data.Primitive.SmallArray` allocates the output `SmallArray (h Any)` in a single pass without intermediate list cells:

```
{-# LANGUAGE AllowAmbiguousTypes #-}
{-# LANGUAGE ConstraintKinds #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE RankNTypes #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE TypeApplications #-}

import Control.Monad.ST (ST)
import Data.Primitive.SmallArray
  ( SmallArray
  , indexSmallArray
  , newSmallArray
  , runSmallArray
  , sizeofSmallArray
  , writeSmallArray
  )
import GHC.Exts (Any)
import Polytropos.Core
  ( AllFields (..)
  , Dict (..)
  , ProductRowT
  , Rep (..)
```

```
  , RowRep (..)
  , productRep
  )

-- | Zip two product RowReps over a constrained rank-2 binary function.
hcliftA2
  :: forall c row f g h a.
     AllFields ProductRowT row c
  => (forall x. c x => f x -> g x -> h x)
  -> RowRep f ProductRowT row a
  -> RowRep g ProductRowT row a
  -> RowRep h ProductRowT row a
hcliftA2 fn (RowRep repF) (RowRep repG) =
  let dicts = fieldDicts @ProductRowT @row @c
      arrF  = repVals repF
      arrG  = repVals repG
      n     = sizeofSmallArray dicts
      arrH  = runSmallArray $ do
        dst <- newSmallArray n (error "hcliftA2: uninitialized slot")
        let go :: Int -> ST s ()
            go i
              | i >= n    = pure ()
              | otherwise =
                  case indexSmallArray dicts i of
                    Dict -> do
                      let fx = indexSmallArray arrF i
                          gx = indexSmallArray arrG i
                      writeSmallArray dst i (fn @Any fx gx)
                      go (i + 1)
        go 0
        pure dst
  in RowRep (Rep 0 arrH)
```

(If you prefer using `Polytropos.Core.productRep`, the inner loop simplifies to building `[h Any]` and wrapping with `RowRep (productRep (go 0))`.)

B. Unconstrained `hliftA2` Requires Zero Constraints

In `generics-sop`, even unconstrained `hliftA2` requires `SListI xs` to know the spine shape. In `polytropos`, the two input RowRep values already carry their runtime length in `repVals`, so unconstrained `hliftA2` needs **no typeclass constraints at all**:

```
-- | Unconstrained heterogeneous binary zip across two product RowReps.
-- Requires no AllFields or SListI constraint.
hliftA2
  :: (forall x. f x -> g x -> h x)
  -> RowRep f ProductRowT row a
  -> RowRep g ProductRowT row a
  -> RowRep h ProductRowT row a
hliftA2 fn (RowRep repF) (RowRep repG) =
  let arrF = repVals repF
      arrG = repVals repG
      n    = sizeofSmallArray arrF
      arrH = runSmallArray $ do
        dst <- newSmallArray n (error "hliftA2: uninitialized slot")
        let go i
              | i >= n    = pure ()
              | otherwise = do
                  let fx = indexSmallArray arrF i
                      gx = indexSmallArray arrG i
                  writeSmallArray dst i (fn @Any fx gx)
                  go (i + 1)
        go 0
        pure dst
  in RowRep (Rep 0 arrH)
```

> **Note on `SumRowT` (Variants):** For unary `hcmap` on a single-payload variant (`RowRep f SumRowT row a`), the active constructor's dictionary lives at `indexSmallArray (fieldDicts @SumRowT @row @c) (fromIntegral (repTag rep))`, and the payload sits at index `0` of `repVals`. For binary `hcliftA2` on `SumRowT`, two variants can only be zipped when `repTag repF == repTag repG`, returning `Maybe (RowRep h SumRowT row a)`.

2. Implementing `hsequence'` and `hsequence`

In `generics-sop`, `hsequence'` sequences an `Applicative` effect `m` out of a composed functor (`m :.: f`) across the product spine, while `hsequence` sequences `NP m xs -> m (NP I xs)`.

In `polytropos`, `Rep` stores `repTag :: !Word` and `repVals :: !(SmallArray (f Any))`. Because `SmallArray` from

`Data.Primitive.SmallArray` already implements `Traversable`, `hsequence'` and `hsequence` have three major advantages over `generics-sop`:

1. **Zero row constraints (`AllFields` / `SListI`) are needed.**
2. **Zero `unsafeCoerce` calls are needed.**
3. **Both functions work polymorphically over `r` (`ProductRowT` and `SumRowT`)**: preserving `repTag` keeps `0` for records and preserves the active constructor tag for variants.

```
{-# LANGUAGE PolyKinds #-}

import Data.Functor.Compose (Compose (..))
import Data.Functor.Identity (Identity (..))
import Polytropos.Core (Rep (..), RowRep (..))

-- | Sequence an Applicative effect out of a Compose-wrapped RowRep.
-- Works for both ProductRowT (records) and SumRowT (variants) with zero constraints.
hsequence'
  :: Applicative m
  => RowRep (Compose m f) r row a
  -> m (RowRep f r row a)
hsequence' (RowRep (Rep tag vals)) =
  RowRep . Rep tag <$> traverse getCompose vals

-- | Sequence an Applicative effect directly from RowRep m into RowRep Identity.
hsequence
  :: Applicative m
  => RowRep m r row a
  -> m (RowRep Identity r row a)
hsequence (RowRep (Rep tag vals)) =
  RowRep . Rep tag <$> traverse (fmap Identity) vals
```

3. Fused `hctraverse` (Combining `hcmap` + `hsequence'`)

In `generics-sop`, idioms like `hsequence' . hcmap p (Comp . f)` allocate an intermediate `NP (m :.: g) xs` structure before sequencing it. In `polytropos`, you can fuse constrained mapping and `Applicative` sequencing into a single `hctraverse` pass over `fieldDicts` and `repVals`:

```
{-# LANGUAGE AllowAmbiguousTypes #-}
{-# LANGUAGE ConstraintKinds #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE RankNTypes #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE TypeApplications #-}

import Data.Primitive.SmallArray
  ( indexSmallArray
  , sizeofSmallArray
  , smallArrayFromListN
  )
import GHC.Exts (Any)
import Polytropos.Core
  ( AllFields (..)
  , Dict (..)
  , ProductRowT
  , Rep (..)
  , RowRep (..)
  )

-- | Fused constrained traversal over a product RowRep without allocating
-- an intermediate Compose SmallArray.
hctraverse
  :: forall c row m f g a.
     (AllFields ProductRowT row c, Applicative m)
  => (forall x. c x => f x -> m (g x))
  -> RowRep f ProductRowT row a
  -> m (RowRep g ProductRowT row a)
hctraverse fn (RowRep (Rep tag arrF)) =
  let dicts = fieldDicts @ProductRowT @row @c
      n     = sizeofSmallArray dicts
      go :: Int -> m [g Any]
      go i
        | i >= n    = pure []
        | otherwise =
```

```
          case indexSmallArray dicts i of
            Dict ->
              (:) <$> fn @Any (indexSmallArray arrF i)
                  <*> go (i + 1)
  in RowRep . Rep tag . smallArrayFromListN n <$> go 0
```

Summary Comparison: `generics-sop` vs. `polytropos`

Combinator	`generics-sop` (NP / SOP)	`polytropos` (`RowRep f r row a`)
`hliftA2`	Requires `SListI xs`; recurses over GADT spine	**Zero constraints**; zips two `SmallArrays` in-place via `runSmallArray`
`hcliftA2`	Requires inductive `All c xs` dictionary tree	Uses $O(1)$ plugin-emitted `fieldDicts` array + `runSmallArray`
`hsequence'`	Requires `SListI xs`; separate `NP` and `SOP` instances	**Zero constraints**, **zero `unsafeCoerce`**, unified across `ProductRowT` & `SumRowT` via `traverse getCompose`
`hsequence`	Derived via `hsequence'` + `hliftA`	Direct `traverse (fmap Identity)` over `repVals`
`hctraverse`	Composed (`hsequence' . hcmap`), allocating intermediate spine	Single fused pass over `fieldDicts` and `repVals`

User prompt: Would these additional things be a good way to perhaps expand `polytropos-generics` or would it make more sense to add a `polytropos-sop` package to provide similar additional interfaces? What further interface functions make sense to add for compatibility with `generics-sop`?

Response: Deciding between expanding `polytropos-generics` and introducing a dedicated `polytropos-sop` package comes down to separating **native `RowRep` array combinators** from **`generics-sop` API compatibility shims**.

1. `polytropos-generics` vs. `polytropos-sop`: Architectural Trade-Offs

In the upstream `generics-sop` ecosystem, functionality is split into two packages: `sop-core` (which defines `NP`, `NS`, `SOP`, `hpure`, `hcliftA2`, `hcollapse`, and `hsequence`) and `generics-sop` (which layers datatype-generic metadata and `Generic` instances on top). Meanwhile, in `polytropos`, `polytropos-core` already defines the unified `Rep` and `RowRep` primitives, and `polytropos-generics` provides `GHC.Generics` bidirectional mappings between nominal types and open rows.

Design Option	Advantages	Drawbacks
Option A: Expand `polytropos-generics` (or `polytropos-core`)	• Zero extra package maintenance overhead. • Combinators like `hpure`, `hcliftA2`, `hcollapse`, and `hsequence` only depend on `base`, `primitive`, and `polytropos-core` (`Rep`, `RowRep`, `AllFields`). • Keeps all generic derivation tools for satellite packages (`polytropos-aeson`, `polytropos-dhall`, `polytropos-cbor`) in one place.	• Conflates `GHC.Generics` tree adapters with flat SOP array combinators. • Naming clashes if you want exact `generics-sop` signatures (which take explicit `Proxy c` arguments) alongside cleaner `TypeApplications`-first `polytropos` signatures.
Option B: Add a dedicated `polytropos-sop` satellite	• Mirrors the `polytropos-lens` / `polytropos-optics` / `polytropos-profunctors` satellite pattern already used in the repo. • Can depend directly on `sop-core` (or re-export its functor combinators `I`, `K`, `:.:`, `-.->`) to offer drop-in signature compatibility (`Proxy c`, `SListI` aliases) for porting existing `generics-sop` code. • Keeps `polytropos-generics` strictly focused on `GHC.Generics` bridging.	• Adds another Cabal package to maintain if it only contains a few hundred lines of array loops.

Recommended Split

1. **Put the `TypeApplications`-native `RowRep` combinators in `polytropos-core` or `polytropos-generics`** (e.g., `Polytropos.Generics.SOP` or `Polytropos.Core.SOP`): Functions like `hcollapse`, `hcpure`, `hcliftA2`, `hctraverse`, and `hsequence` are foundational operations on `RowRep` that require zero external dependencies beyond `primitive` and `base`. Every satellite package in `polytropos` benefits from having them available without pulling in extra dependencies.

2. **Create `polytropos-sop` only if you want a `sop-core` compatibility/bridge layer**: If you want `Proxy`-passing signatures (`hcliftA2 (Proxy @Show) ...`), `sop-core` functor re-exports (`I`, `K`, `:.:`, `-.->`), or conversion bridges between `Generics.SOP.NP/NS` and `Polytropos.Core.RowRep`, isolate those in `polytropos-sop`.

2. Further `generics-sop` Interface Functions to Add

Because `polytropos` represents both products (`ProductRowT`) and sums (`SumRowT`) with the same flat Rep (`!Word` tag + `SmallArray (f Any)`), several `generics-sop` combinators become either **polymorphic over products and sums simultaneously** or $O(1)$ **instead of** $O(N)$.

A. Unified Product & Sum Mapping and Folding (`hcmap`, `hcfoldMap`)

In `generics-sop`, `hcmap` must be defined separately for `NP` (iterating all elements) and `NS` (chasing `S` constructors to the active branch), and folding requires allocating an intermediate `NP (K m) xs` before calling `hcollapse`.

In `polytropos`:

- **`hcmap` works on both `ProductRowT` and `SumRowT`**: For `ProductRowT`, slot `i` uses dictionary `i`; for `SumRowT`, the single active payload at slot `0` uses the dictionary at index `repTag` in `fieldDicts`.
- **`hcfoldMap` fuses `mconcat . hcollapse . hcmap`**: It accumulates into a `Monoid m` directly across `repVals` without allocating an intermediate `SmallArray (Const m Any)`.

```
-- | Map a constrained rank-2 function over a SumRowT RowRep in O(1) time.
hcmapSum
  :: forall c row f g a.
     AllFields SumRowT row c
  => (forall x. c x => f x -> g x)
  -> RowRep f SumRowT row a
  -> RowRep g SumRowT row a
hcmapSum fn (RowRep (Rep tag vals)) =
  let dicts = fieldDicts @SumRowT @row @c
      !d    = indexSmallArray dicts (fromIntegral tag)
      !v    = indexSmallArray vals 0
      !v'   = case d of Dict -> fn @Any v
  in RowRep (Rep tag (smallArrayFromListN 1 [v']))

-- | Fused constrained fold over a ProductRowT RowRep (avoids intermediate Const array).
hcfoldMap
  :: forall c row m f a.
     (AllFields ProductRowT row c, Monoid m)
  => (forall x. c x => f x -> m)
  -> RowRep f ProductRowT row a
  -> m
hcfoldMap fn (RowRep (Rep _ vals)) =
  let dicts = fieldDicts @ProductRowT @row @c
      n     = sizeofSmallArray dicts
      go i
        | i >= n    = mempty
        | otherwise =
            case indexSmallArray dicts i of
              Dict -> fn @Any (indexSmallArray vals i) <> go (i + 1)
  in go 0
```

B. Function Application (`hap`) & Product-Sum Elimination (`helim`)

In `generics-sop`, the lifted function arrow is `newtype (-.->) f g a = Fn { apFn :: f a -> g a }`, and `hap` applies a structure of functions to a structure of arguments:

```
newtype (-.->) f g a = Fn { apFn :: f a -> g a }
infixr 1 -.->
```

In `polytropos`, hap provides **three** operations—all requiring **zero typeclass constraints**:

1. **Product-to-Product (`hapProduct`)**: Pointwise zip via `hliftA2 apFn`.
2. **Product-of-Functions to Sum-of-Payloads (`hapSum`)**: Uses the variant's unpacked `repTag` to index directly into the product of functions in $O(1)$ **time**.
3. **Direct Sum Elimination (`helim` / `ana` dual)**: Collapses `hapSum` + `hcollapse` into a single $O(1)$ array lookup when eliminating a `SumRowT` with a `ProductRowT` of handlers returning a uniform type `r`.

```
-- | Apply a ProductRowT of lifted functions to a ProductRowT of values (O(n), 0 constraints).
hapProduct
  :: RowRep (f -.-> g) ProductRowT row a
  -> RowRep f          ProductRowT row a
  -> RowRep g          ProductRowT row a
hapProduct = hliftA2 apFn

-- | Apply a ProductRowT of branch functions to a SumRowT variant in O(1) time!
hapSum
  :: RowRep (f -.-> g) ProductRowT row a
  -> RowRep f          SumRowT     row a
  -> RowRep g          SumRowT     row a
hapSum (RowRep (Rep _ fnArr)) (RowRep (Rep tag valArr)) =
  let fn  = indexSmallArray fnArr (fromIntegral tag)
      val = indexSmallArray valArr 0
  in RowRep (Rep tag (smallArrayFromListN 1 [apFn fn val]))

-- | Eliminate a SumRowT variant using a ProductRowT of handlers in O(1) time.
helim
  :: RowRep (f -.-> Const b) ProductRowT row a
  -> RowRep f                SumRowT     row a
  -> b
helim (RowRep (Rep _ fnArr)) (RowRep (Rep tag valArr)) =
  let fn  = indexSmallArray fnArr (fromIntegral tag)
      val = indexSmallArray valArr 0
  in getConst (apFn fn val)
```

C. $O(1)$ Sum Combinators: `hindex`, `injections`, `apInjs_NP`, and `hexpand`

In `generics-sop`, sum operations on `NS` walk unary Peano tags (`Z`, `S`), making index lookup and sum-to-product expansion $O(N)$. In `polytropos`, `Rep` stores `repTag :: !Word` directly:

- **`hindex`**: Returns the 0-based active branch index of a `SumRowT` in $O(1)$ time.
- **`hexpand`**: Expands a `SumRowT` into a `ProductRowT` of `Compose Maybe f` (where the active tag's slot is `Just` and all other slots are `Nothing`). Using `newSmallArray n (Compose Nothing)` followed by a single `writeSmallArray` at `repTag`, this runs in a single flat allocation.
- **`injections`**: Produces a `RowRep (f -.-> Const (RowRep f SumRowT row a)) ProductRowT row a` where each slot `i` holds the constructor injection function for tag `i`.
- **`apInjs`**: Takes a `RowRep f ProductRowT row a` and produces `[RowRep f SumRowT row a]`, pairing each slot `i` in the product with its constructor tag `i`.

```
-- | O(1) active constructor index of a SumRowT RowRep.
hindex :: RowRep f SumRowT row a -> Int
hindex (RowRep (Rep tag _)) = fromIntegral tag

-- | Expand a SumRowT into a ProductRowT with Just at the active tag and Nothing elsewhere.
hexpand
  :: forall row f a.
     FieldNames row -- or known arity via AllFields
  => RowRep f SumRowT row a
  -> RowRep (Compose Maybe f) ProductRowT row a
hexpand (RowRep (Rep tag valArr)) =
  let n   = length (fieldNames @row)
      val = indexSmallArray valArr 0
      arr = runSmallArray $ do
        dst <- newSmallArray n (Compose Nothing)
        writeSmallArray dst (fromIntegral tag) (Compose (Just val))
        pure dst
  in RowRep (Rep 0 arr)

-- | Convert each slot of a ProductRowT into its corresponding SumRowT variant.
apInjs :: RowRep f ProductRowT row a -> [RowRep f SumRowT row a]
apInjs (RowRep (Rep _ arr)) =
  let n = sizeofSmallArray arr
      go i
        | i >= n    = []
        | otherwise =
            let v = indexSmallArray arr i
            in RowRep (Rep (fromIntegral i) (smallArrayFromListN 1 [v])) : go (i + 1)
  in go 0
```

D. Zero-Copy Coercions (`hcoerce`, `hfromI`, `htoI`)

In `generics-sop`, `hcoerce`, `hfromI`, and `htoI` must traverse the `NP`/`NS` spine or rely on unsafe rules because GADT indices tie

each node to its element type.

In `polytropos`, every slot in `RowRep f r row a` has the erased type `f Any`. Therefore, whenever `forall x. Coercible (f x) (g x)` holds (which means `Coercible (f Any) (g Any)` holds, as with `Identity`, `Const b`, or any newtype wrapper), converting `RowRep f r row a` to `RowRep g r row a` is a **true** $O(1)$ **zero-allocation coercion** that reuses the underlying `SmallArray` pointer without copying a single word:

```
{-# LANGUAGE QuantifiedConstraints #-}

import Data.Coerce (Coercible)
import Unsafe.Coerce (unsafeCoerce)

-- | O(1) zero-copy coercion between representationally identical slot functors
-- (e.g., Identity <-> I, Const a <-> K a, or newtype wrappers).
hcoerce
  :: (forall x. Coercible (f x) (g x))
  => RowRep f r row a
  -> RowRep g r row a
hcoerce = unsafeCoerce
```

E. Metadata Reification (`hfieldNames` / `RowInfo`)

A signature idiom in `generics-sop` is constructing a product of field metadata (`NP (K String) xs` or `NP FieldInfo xs`) and zipping it with a product of values via `hcliftA2` to write generic serializers, SQL encoders, or CLI parsers.

Using `FieldNames row` from `Polytropos.Core`, you can expose `hfieldNames` directly as a `Const String`-wrapped RowRep so it plugs straight into `hliftA2` and `hcliftA2`:

```
-- | Reify the row's field/constructor labels as a Const-wrapped ProductRowT RowRep,
-- ready to be zipped with values via hliftA2 or hcliftA2.
hfieldNames
  :: forall row a.
     FieldNames row
  => RowRep (Const String) ProductRowT row a
hfieldNames =
  let names = map Const (fieldNames @row)
  in RowRep (productRep names)
```

Summary Checklist of `generics-sop` Combinators for `polytropos`

Category	Functions	`polytropos` Advantage over `generics-sop`
Population	`hpure`, `hcpure`	$O(1)$ plugin-synthesized `fieldDicts` array
Mapping & Zipping	`hmap`, `hcmap`, `hliftA2`, `hcliftA2`, `hliftA3`, `hcliftA3`	Unconstrained `hliftA*` needs **zero constraints**; `hcmap` works on both `ProductRowT` & `SumRowT`
Application & Elimination	`hapProduct`, `hapSum`, `helim`	`hapSum` and `helim` dispatch a product of handlers against a sum variant in $O(1)$ **time** via `repTag`
Folding & Collapsing	`hcollapse`, `hcfoldMap`	`hcollapse` needs **zero constraints** and **zero `unsafeCoerce`**; `hcfoldMap` fuses away the intermediate `Const` array
Traversing & Sequencing	`hsequence`, `hsequence'`, `hctraverse`	`hsequence` & `hsequence'` work uniformly on `ProductRowT` and `SumRowT` with **zero constraints**
Sum Operations	`hindex`, `hexpand`, `injections`, `apInjs`	$O(1)$ tag read (`repTag`) instead of $O(N)$ Peano S/Z GADT traversal
Coercion & Metadata	`hcoerce`, `hfromI`, `htoI`, `hfieldNames`	`hcoerce` is a **zero-copy** $O(1)$ **cast** on `RowRep` without copying the `SmallArray`
